

Estimating the Distance of a Quantum LDPC Code by Decoder Simulation: the Syndrome-Decoder Upper Bound, with Sub-Threshold Scaling as Support

June 22, 2026

Abstract

When a CSS quantum LDPC code is too large or too unstructured to certify its distance d exactly (Brouwer–Zimmermann and integer programming both become intractable for dense, high-rate codes), the recognised tool is a decoder-based *heuristic* that returns an *upper bound*. The standard such method is the *syndrome-decoder method*: inject random errors, decode, and read off the weight of the residual logical operator; the minimum over many trials is a checkable upper bound $\hat{d}_{\text{ub}} \geq d$, tightening monotonically with the number of trials. We describe this method, its confidence model, and how it is verified by independent re-simulation. As supporting context we include the sub-threshold logical-error-rate scaling $P_L \sim (p/p_{\text{th}})^{\lceil d/2 \rceil}$, whose slope measures the decoder’s *effective* distance d_{eff} ; this is the standard fault-tolerance suppression law, but $d_{\text{eff}} \leq d$ and can *underestimate* the true distance for a sub-optimal decoder, so it characterises performance rather than finding the code distance. A worked $[[180, 20, 14]]$ example exhibits all signals—including a cautionary case where the effective distance reads 12 while the code distance is 14.

1 Setup and notation

A CSS code on n qubits is given by parity-check matrices $H_X \in \mathbb{F}_2^{m_X \times n}$, $H_Z \in \mathbb{F}_2^{m_Z \times n}$ with $H_X H_Z^T = 0$. Write $C_X = \ker H_Z$, $C_Z = \ker H_X$. The nontrivial X -logicals are $C_X \setminus \text{rowsp } H_X$ and

$$d_X = \min\{\text{wt}(v) : v \in C_X \setminus \text{rowsp } H_X\}, \quad d_Z = \min\{\text{wt}(v) : v \in C_Z \setminus \text{rowsp } H_Z\}, \quad (1)$$

with $d = \min(d_X, d_Z)$. Fix a Z -logical basis $L_Z \in \mathbb{F}_2^{k \times n}$ (computed once from H_X, H_Z by Gaussian elimination); a vector $r \in C_X$ is a nontrivial X -logical iff $L_Z r \neq 0 \pmod{2}$. We probe d_X with an X -error channel; d_Z is symmetric (swap $X \leftrightarrow Z$). “The distance” below means whichever sector is simulated; report $d = \min(d_X, d_Z)$.

Noise model. Code-capacity: each qubit independently suffers an X error with probability p , perfect syndrome measurement. This isolates the code from syndrome-extraction-circuit details. (Circuit-level noise measures a *circuit* effective distance, a different quantity; see Section 6.)

Algorithm 1 Syndrome-decoder distance upper bound

Require: checks H_Z , logical basis L_Z , decoder $\mathcal{D}$, trials T

```

1:  $\hat{d}_{\text{ub}} \leftarrow \infty$ 
2: for  $t = 1, \dots, T$  do
3:   sample an error  $e$ ;  $\hat{c} \leftarrow \mathcal{D}(H_Z e \bmod 2)$ ;  $r \leftarrow e \oplus \hat{c}$ 
4:   if  $L_Z r \neq 0 \pmod{2}$  and  $\text{wt}(r) < \hat{d}_{\text{ub}}$  then  $\hat{d}_{\text{ub}} \leftarrow \text{wt}(r)$ , store  $r$ 
5:   end if
6: end for
7: return  $\hat{d}_{\text{ub}}$  and its witness  $r$   $\triangleright d \leq \hat{d}_{\text{ub}}$ , re-checkable

```

2 The syndrome-decoder distance estimate

2.1 Method

Sample an error e , decode it, and inspect the residual. The syndrome is $\sigma = H_Z e \bmod 2$; a decoder $\mathcal{D}$ returns a correction $\hat{c}$ with $H_Z \hat{c} = \sigma$, so the residual

$$r = e \oplus \hat{c} \in C_X = \ker H_Z \quad (2)$$

always lies in the X -code. Whenever $L_Z r \neq 0 \pmod{2}$, the residual is a genuine nontrivial X -logical, and therefore an explicit witness of weight $\text{wt}(r) \geq d$. Tracking the minimum such weight over many trials gives

$$\hat{d}_{\text{ub}} = \min_{\text{trials with } L_Z r \neq 0} \text{wt}(r) \geq d, \quad (3)$$

a *checkable upper bound* on the distance: each candidate is a concrete operator one can re-verify ($L_Z r \neq 0$ and $H_Z r = 0$). This is the *syndrome-decoder method*, a recognised heuristic distance-finder for large quantum LDPC codes [1, 3]; it is the decoder-based cousin of the random-information-set search of QDistRnd [2].

Decoder. qLDPC checks have weight > 2 , so the code is not matchable and minimum-weight perfect matching does not apply. Use **BP + OSD** (belief propagation with ordered-statistics decoding) [3, 4]; a reference implementation is the `ldpc` package. A higher OSD order finds lower-weight residuals (tighter $\hat{d}_{\text{ub}}$); report the order used.

Error distribution. Any distribution that drives the decoder into nontrivial cosets works; two useful choices are (i) code-capacity errors at a rate near threshold, which produce a steady stream of residual logicals, and (ii) fixed-weight errors of weight $\approx \lceil d/2 \rceil$, which target the regime where low-weight logicals appear. The estimate (3) is an upper bound under any choice.

2.2 Confidence

$\hat{d}_{\text{ub}}$ is a non-increasing function of the trial count T that is bounded below by d , so it converges from above. Confidence is the *saturation* of this running minimum: once $\hat{d}_{\text{ub}}$ has not decreased over a large number of additional trials, low-weight logicals are being missed only with small probability. Two quantitative handles:

- *Hitting probability.* If a weight- w logical is reachable as a residual with per-trial probability q , the chance of missing it in T trials is $(1 - q)^T \leq e^{-qT}$; observing no improvement over T trials bounds the missed mass by the rule of three, $q \lesssim 3/T$ at 95%.

- *Cross-method agreement.* Running an independent upper-bound search (QDistRnd / random information set [2], or QDistEvol [1]) and obtaining the same value strengthens the claim that $\hat{d}_{\text{ub}} = d$.

The method returns an *upper* bound; a matching *lower* bound (hence a certificate $d = \hat{d}_{\text{ub}}$) requires an exact method—Brouwer–Zimmermann, MaxSAT, or integer programming [1]—which is feasible only for low-rate or structured codes (Section 6).

3 Supporting context: effective distance from sub-threshold scaling

The logical error rate of a distance- d code under a good decoder obeys the standard fault-tolerance suppression law

$$P_L(p) \sim A p^{\lceil d/2 \rceil} \quad (p \rightarrow 0), \quad \Longleftrightarrow \quad \log P_L = \lceil d/2 \rceil \log p + \log A + O(p), \quad (4)$$

because a decoder that corrects all errors of weight $\leq \lfloor (d-1)/2 \rfloor$ can only fail at weight $\geq \lceil d/2 \rceil$, and the lowest surviving power of p dominates as $p \rightarrow 0$. The slope of $\log P_L$ versus $\log p$ thus tends to $\lceil d/2 \rceil$. This is the law behind exponential error suppression and the Google “below threshold” experiments ($\varepsilon_L \propto \Lambda^{-(d+1)/2}$) [5, 6].

3.1 It measures the *effective*, not the code, distance

The exponent recovered from a fit is set by the *decoder*, not by the code geometry alone. Define the effective distance d_{eff} by $\lceil d_{\text{eff}}/2 \rceil =$ asymptotic slope. A maximum-likelihood decoder gives $d_{\text{eff}} = d$; a sub-optimal decoder that occasionally fails on an error of weight $< \lceil d/2 \rceil$ introduces a lower power of p and *lowers* the slope, so

$$d_{\text{eff}} \leq d. \quad (5)$$

This gap is documented and can be large: approximate decoders have been reported at effective distances of only 43–65% of the true distance [6]. Consequently the slope characterises *performance* and yields a *lower-leaning* estimate of d ; it is not, by itself, a reliable code-distance finder, and the distance-finding literature does not list it among distance algorithms [1].

3.2 Fitting and its statistics

With $x_i = \log p_i$, $y_i = \log \widehat{P}_L(p_i)$ and failure counts f_i , the delta method gives $\text{Var}(y_i) \approx 1/f_i$, so a weighted least-squares fit with weights $w_i = f_i$ yields slope $\hat{\beta}$ and

$$\text{SE}(\hat{\beta}) = \left[\sum_i f_i (x_i - \bar{x}_w)^2 \right]^{-1/2}, \quad d_{\text{eff}} \approx 2\hat{\beta} \quad (\pm 2 \text{SE}(\hat{\beta}) \text{ at } 1\sigma). \quad (6)$$

Two practical hazards: (i) finite- p curvature makes the local slope *rise* toward $\lceil d_{\text{eff}}/2 \rceil$ as p decreases, so fit only the saturated window $p \in [p_{\text{th}}/6, p_{\text{th}}/3]$; and (ii) under-resolved points (few f_i) must be dropped—they both inflate SE and, if included, bias the fit. To shrink the interval, increase the failures f_i (adaptive sampling to a fixed failure count) and widen the $\log p$ lever arm by placing points at the extremes of the valid window.

3.3 The error-weight onset: the sharpest probe of d_{eff} (and a warning)

A direct, curvature-free way to read $\lceil d_{\text{eff}}/2 \rceil$ is to estimate $q_w = \Pr(\text{failure} \mid \text{error of weight exactly } w)$ for increasing w : q_w jumps from 0 to positive at $w^* = \lceil d_{\text{eff}}/2 \rceil$, and a zero-failure run at $w^* - 1$ gives $q_{w^*-1} \leq 3/N$ (95%). This is sharper and cheaper than the slope fit, but it keys on the *lowest* weight at which the decoder *ever* fails and is therefore the most sensitive to sub-optimality (5): it returns d_{eff} , which may sit well below d . The discriminator is to inspect the residual weights of those onset failures (Section 2): if the decoder’s lowest-weight failures still land on *high*-weight logicals, the low onset is decoder sub-optimality, not a light logical—see the worked example.

4 Speed and scale

Both methods are Monte Carlo and embarrassingly parallel: independent trials with distinct seeds, aggregated by a minimum (syndrome decoder) or by summed counts (slope). Profiling BP + OSD on the worked-example code (single core): decode-only $\approx 5.0 \times 10^4/\text{s}$, full sampling loop $\approx 2.6 \times 10^4/\text{s}$ —so roughly half the single-core wall-time is Python overhead around an already-native (C++) decoder. The two effective levers, with measured speed-ups on an 11-core machine:

Multiprocessing (the dominant, near-free lever). Run independent worker processes, each with its own decoder and seed; aggregate. For the syndrome-decoder method this took the throughput from 1.7×10^4 to 1.46×10^5 trials/s, an **8.6×** speed-up on 9 workers, and the extra trials tightened $\hat{d}_{\text{ub}}$ from 15 to the correct 14. One platform caveat dominates the result: use the `fork` start method, not `spawn`. Under `spawn` (the macOS default) every worker re-imports NumPy and the decoder library, which halved the speed-up to $\sim 4.5\times$; `fork` inherits the parent image and recovers the near-linear scaling.

Threaded native kernel (a C++ port done cheaply). The random-information-set upper bound (the QDistRnd-style search) is pure linear algebra and is already a C++ extension; making it multi-threaded is a small change with *no decoder-quality risk*. Splitting the trials across `std::threads` per side—over a self-contained, const-input core routine, so no locks—and releasing the GIL ran at 1.27×10^5 trials/s, **7.4×** over one core, in a single Python call with none of the pickling or process-startup cost of multiprocessing, and still returns the correct $\hat{d}_{\text{ub}}$. This 7.4× is the *hardware ceiling*, not a software limit: the test machine has 5 performance and 6 efficiency cores, and the efficiency cores run at roughly a third of the speed, so the available throughput is $\approx 5 + 6 \times 0.4 \approx 7.4$ performance-core-equivalents—saturated by using all 11 threads (fewer leaves cores idle; more does not help). Two refinements gave little here: hoisting the per-thread basis recompute (the basis is cheap next to the trials loop) and load-balancing the static split (the heterogeneous cores, not the split, set the ceiling). The more compute-bound BP + OSD multiprocessing reached a slightly higher 8.6× because its workload draws more fully on the efficiency cores; the achievable factor is workload-dependent, but “use all the cores” is the rule.

What not to do. Re-implementing BP + OSD in C++ to remove the remaining Python overhead is *not* worthwhile: the decoder is already native, so a rewrite reclaims only the $\sim 2\times$ glue factor while risking a weaker OSD that returns *looser* residual bounds—degrading the very estimate it is meant to speed up. Prefer multiprocessing around the good decoder, and thread the native QDistRnd kernel for the random-information-set bound.

Scaling and the rare-event wall. The syndrome-decoder method needs only enough trials for $\hat{d}_{\text{ub}}$ to saturate, whereas the slope method’s sub-threshold cost grows as $p^{-\lceil d/2 \rceil}$: even a $20\times$ engine extends the reachable p -window only modestly, a further argument for the upper-bound methods when d is large.

5 Verification by independent re-simulation

A claim of this kind is verified by *re-running the experiment*, not by re-checking the claimant’s arithmetic. Given only the code (H_X, H_Z , from which L_Z is recomputed) plus the reported decoder configuration and seed, an independent party reconstructs the harness and reproduces the estimate: for the syndrome-decoder bound, confirm a residual logical of weight $\leq \hat{d}_{\text{ub}}$ is found (and, ideally, re-verify the stored witness directly: $H_Z r = 0, L_Z r \neq 0$); for the slope, rerun the sweep and confirm the saturated slope. The submitted counts and fitted numbers are *reproducibility metadata*, not the object of trust. Re-simulation is minutes-scale, which is acceptable: it is the honest cost of an independent statistical check, and exact certification, where feasible at all, is itself minutes-to-hours.

6 What the method does and does not establish

- The syndrome-decoder method returns a checkable *upper* bound $d \leq \hat{d}_{\text{ub}}$ (3), with confidence given by the saturation of the running minimum and by agreement with independent upper-bound searches (QDistRnd/QDistEvol). It is *not* a certificate: a matching lower bound needs an exact method (Brouwer–Zimmermann, MaxSAT, integer programming), feasible only for low-rate or structured codes.
- The slope (and the error-weight onset) measure the *effective* distance $d_{\text{eff}} \leq d$ (5); with a near-ML decoder $d_{\text{eff}} = d$, but a sub-optimal decoder can read substantially lower. Use them for performance characterisation and as corroboration, not as the primary code-distance estimate.
- Code-capacity quantities are properties of the *code*. Circuit-level memory experiments (e.g. in Stim) measure a circuit effective distance that folds in the syndrome-extraction schedule and is generally smaller; use those for fault-tolerance performance.
- These simulations are the statistical complement to exact distance computation. They are most useful exactly where the exact methods fail: dense, high-rate codes whose minimum-weight-logical ILP has a loose relaxation and whose information sets are too large to enumerate.

A Worked example: $[[180, 20, 14]]$

A weight-8 two-block (coset) qLDPC code, X -sector, BP + OSD decoder. The three signals illustrate the hierarchy of this note.

Syndrome-decoder upper bound (primary). Random errors decoded; the minimum residual logical weight observed is

$$\hat{d}_{\text{ub}} = 14,$$

in agreement with an independent QDistRnd-style random-information-set search that returns 14 and does not improve over 3×10^5 trials. Hence $d \leq 14$ with high confidence (two independent upper-bound methods saturate at 14).

Slope (supporting). Adaptive sampling to $F \approx 60$ failures, threshold $p_{\text{th}} \approx 0.085$:

p	$\widehat{P}_{\text{L}}$	f/N	local slope
0.050	5.1×10^{-2}	102/2,000	—
0.040	1.2×10^{-2}	73/6,000	6.4
0.030	1.6×10^{-3}	60/38,000	7.1

The weighted fit gives slope 6.8 ± 0.3 , i.e. $d_{\text{eff}} \approx 13.6$ (95% CI [12.3, 14.8])—consistent with 14 in this window.

Error-weight onset (cautionary). Fixed-weight sampling, OSD order 6–20, 10^6 samples per weight:

$$q_5 = 0 (\leq 3 \times 10^{-6}), \quad q_6 \approx 1 \times 10^{-5}, \quad q_7 \approx 4 \times 10^{-5},$$

so the onset is $w^* = 6$, giving $d_{\text{eff}} \in \{11, 12\}$ —*below* the true distance. Raising the OSD order from 6 to 20 barely moved q_6 , confirming this is decoder sub-optimality, not a near-threshold artefact. The discriminator clinches it: the residual logicals produced by those weight-6 failures have weight 18–31 (minimum 18), and the lightest residual seen at any onset weight is 14. No logical lighter than 14 is ever exhibited. The onset’s “12” is the effective distance; the code distance is 14.

Takeaway. The upper-bound (syndrome-decoder / QDistRnd) value 14 is the trustworthy distance estimate; the slope corroborates it; the error-weight onset measures the BP + OSD effective distance (≈ 12) and, taken alone, would have underestimated d . Always cross-check a low onset against the residual weights.

B A family of estimated codes

The primary (syndrome-decoder) method, multiprocessed across all cores and cross-checked against the threaded QDistRnd bound, applied to a saved family of weight-8 coset codes on the same group ($n = 180$). Each $\hat{d}_{\text{ub}}$ is the minimum residual logical weight over 6×10^5 trials per method; the two independent upper-bound methods agree on every code.

code	n	k	$\hat{d}_{\text{ub}}$ (synd.)	$\hat{d}_{\text{ub}}$ (QDistRnd)	kd^2/n
coset_180_20_14_0	180	20	14	14	21.8
coset_180_18_14_1	180	18	14	14	19.6
coset_180_16_14_{2..5}	180	16	14	14	17.4

Every code carries an explicit weight-14 witness operator, so the values are upper bounds: the estimated parameters are $[[180, k, \leq 14]]$, not certified distances. The efficiencies kd^2/n assume $d = 14$ (consistent with both methods saturating there and with the slope corroboration of Appendix A); they would beat the standard bivariate-bicycle frontier of ≈ 13.5 if the distance is confirmed.

References

- [1] *Distance-Finding Algorithms for Quantum Codes and Circuits*, arXiv:2603.22532 (2026). Taxonomy of exact (Brouwer–Zimmermann, MIP, MaxSAT, connected-cluster) and heuristic (random information set / QDistRnd, QDistEvol, BP + OSD syndrome decoder, Stim undetectable-error) methods.
- [2] L. P. Pryadko et al., *QDistRnd: A GAP package for computing the distance of quantum error-correcting codes*, J. Open Source Softw. **7**, 4120 (2022). Random-information-set upper bound.
- [3] J. Roffe, D. R. White, S. Burton, E. Campbell, *Decoding across the quantum low-density parity-check code landscape*, Phys. Rev. Research **2**, 043423 (2020); `ldpc` package.
- [4] P. Panteleev, G. Kalachev, *Degenerate quantum LDPC codes with good finite length performance*, Quantum **5**, 585 (2021). Ordered-statistics decoding.
- [5] E. Dennis, A. Kitaev, A. Landahl, J. Preskill, *Topological quantum memory*, J. Math. Phys. **43**, 4452 (2002). Sub-threshold scaling.
- [6] Google Quantum AI, *Quantum error correction below the surface code threshold*, Nature **638**, 920 (2025); arXiv:2408.13687. Λ -suppression and effective distance.
- [7] C. Gidney, *Stim: a fast stabilizer circuit simulator*, Quantum **5**, 497 (2021). Circuit-level / undetectable-error methods.